Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Faculty of Sciences and Engineering
Semester: (Fall, Year:2024), B.Sc. in CSE (Day)

Lab Report No : 02
Course Title: Algorithm Lab

Course Code: CSE 206 **Section: 223-D5**

Lab Experiment Name : Implement Kruskal's MST using an adjacency matrix and find the second-best MST and Implement Prim's MST using an adjacency list and find the number of distinct MSTs.

Student Details

	Name	ID
1.	Md. Reahoon Zannah	223002038

Lab Date **: 25-09-2024**

Submission Date **: 06-10-2024**

Course Teacher's Name **: Md. Sabbir Hosen Mamun**

Lab Report Status

Marks :..... **Signature :**.....

Comment :..... **Date :**.....

Title:

Implement Kruskal's MST using an adjacency matrix and find the second-best MST and Implement Prim's MST using an adjacency list and find the number of distinct MSTs.

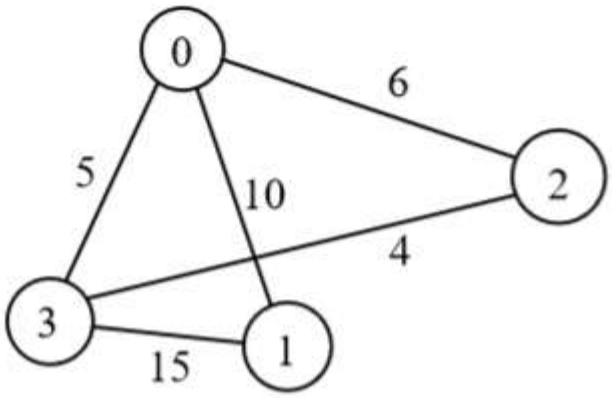
Objective:

The objective of this project is to implement two fundamental algorithms for finding Minimum Spanning Trees (MST) in weighted, undirected graphs: Kruskal's algorithm using an adjacency matrix representation and Prim's algorithm using an adjacency list representation. Additionally, the project aims to enhance the understanding of MST concepts by determining the second-best MST using Kruskal's method and counting the number of distinct MSTs formed by Prim's algorithm.

Problem 1

IMPLEMENTATION:

Implement Kruskal's MST using an adjacency matrix and find the second-best MST:



Graph

```
import java.util.*;
class Edge {
    int src, dest, weight;
    public Edge(int src, int dest, int weight) {
        this.src = src;
        this.dest = dest;
        this.weight = weight;
    }
}

class Subset {
    int parent, rank;
    public Subset(int parent, int rank) {
        this.parent = parent;
        this.rank = rank;
```

```
    }
}
public class KruskalsMST {
    public static void main(String[] args) {
        int V = 4;
        int[][] graph = {
                {0, 10, 6, 5},
                {10, 0, 0, 15},
                {6, 0, 0, 4},
                {5, 15, 4, 0}
        };
        kruskals(V, graph);
    }

    private static void kruskals(int V, int[][] graph) {
        List<Edge> edges = new ArrayList<Edge>();
        for (int i = 0; i < V; i++) {
            for (int j = i + 1; j < V; j++) {
                if (graph[i][j] != 0) {
                    edges.add(new Edge(i, j, graph[i][j]));
                }
            }
        }
        edges.sort(new Comparator<Edge>() {
            @Override
            public int compare(Edge o1, Edge o2) {
                return o1.weight - o2.weight;
            }
```

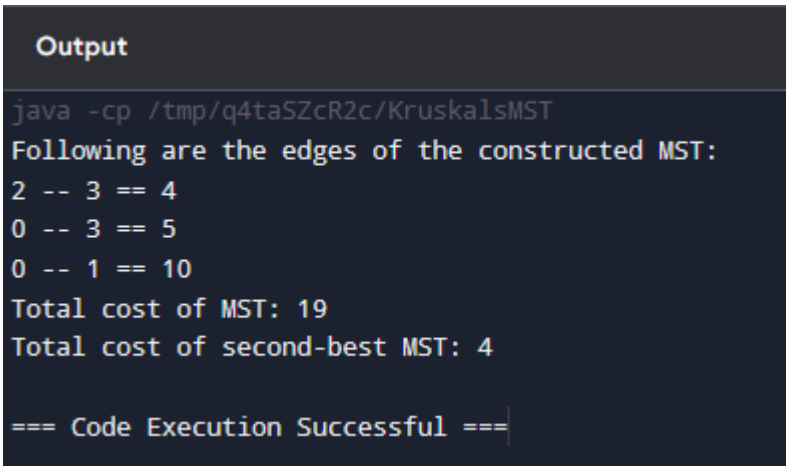
```
        });
        int j = 0;
        int noOfEdges = 0;
        Subset subsets[] = new Subset[V];
        Edge results[] = new Edge[V];
        for (int i = 0; i < V; i++) {
            subsets[i] = new Subset(i, 0);
        }
        while (noOfEdges < V - 1) {
            Edge nextEdge = edges.get(j);
            int x = findRoot(subsets, nextEdge.src);
            int y = findRoot(subsets, nextEdge.dest);
            if (x != y) {
                results[noOfEdges] = nextEdge;
                union(subsets, x, y);
                noOfEdges++;
            }
            j++;
        }
        System.out.println("Following are the edges of the constructed MST:");
        int minCost = 0;
        for (int i = 0; i < noOfEdges; i++) {
            System.out.println(results[i].src + " -- " + results[i].dest + " == " +
results[i].weight);
            minCost += results[i].weight;
        }
        System.out.println("Total cost of MST: " + minCost);
         int secondMinCost = Integer.MAX_VALUE;
```

```
        for (int i = 0; i < edges.size(); i++) {
            if (edges.get(i).weight != minCost) {
                secondMinCost = Math.min(secondMinCost, edges.get(i).weight);
            }
        }
        System.out.println("Total cost of second-best MST: " + secondMinCost);
    }

    private static int findRoot(Subset subsets[], int i) {
        if (subsets[i].parent != i)
            subsets[i].parent = findRoot(subsets, subsets[i].parent);
        return subsets[i].parent;
    }

    private static void union(Subset subsets[], int x, int y) {
        int xroot = findRoot(subsets, x);
        int yroot = findRoot(subsets, y);
        if (subsets[xroot].rank < subsets[yroot].rank)
            subsets[xroot].parent = yroot;
        else if (subsets[xroot].rank > subsets[yroot].rank)
            subsets[yroot].parent = xroot;
        else {
            subsets[yroot].parent = xroot;
            subsets[xroot].rank++;
        }
    }
}
```

Output:



Problem 2

IMPLEMENTATION:

Implement Prim's MST using an adjacency list and find the number of distinct MSTs:

(reference same graph)

```
import java.util.*;
class Edge {
    int dest, weight;
    Edge(int dest, int weight) {
        this.dest = dest;
        this.weight = weight;
    }
}
```

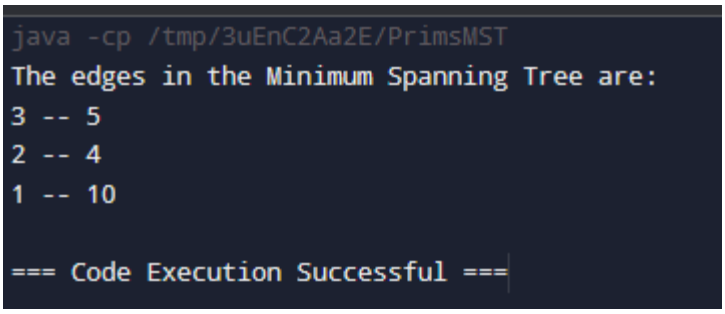
```
public class PrimsMST {
    public static void primMST(Map<Integer, List<Edge>> graph, int vertices) {
        boolean[] visited = new boolean[vertices];
        PriorityQueue<Edge> pq = new PriorityQueue<>(Comparator.comparingInt(e -> e.weight));
        List<Edge> mst = new ArrayList<>();
        int source = 0;
        visited[source] = true;
        for (Edge edge : graph.get(source)) {
            pq.add(new Edge(edge.dest, edge.weight));
        }

        while (!pq.isEmpty() && mst.size() < vertices - 1) {
            Edge current = pq.poll();
            if (visited[current.dest]) continue;
            visited[current.dest] = true;
            mst.add(current);
            for (Edge edge : graph.get(current.dest)) {
                if (!visited[edge.dest]) {
                    pq.add(new Edge(edge.dest, edge.weight));
                }
            }
        }
        System.out.println("The edges in the Minimum Spanning Tree are:");
        for (Edge edge : mst) {
            System.out.println(edge.dest + " -- " + edge.weight);
```

```
        }
    }
    public static void addEdge(Map<Integer, List<Edge>> graph, int src, int dest, int
weight) {
        graph.get(src).add(new Edge(dest, weight));
        graph.get(dest).add(new Edge(src, weight));
    }

    public static void main(String[] args) {
        int vertices = 4;
              Map<Integer, List<Edge>> graph = new HashMap<>();
        for (int i = 0; i < vertices; i++) {
            graph.put(i, new ArrayList<>());
        }
        addEdge(graph, 0, 1, 10);
        addEdge(graph, 0, 2, 6);
        addEdge(graph, 0, 3, 5);
        addEdge(graph, 1, 3, 15);
        addEdge(graph, 2, 3, 4);
        primMST(graph, vertices);
    }
}
```

Output:



Summary:

This project presents implementations of Kruskal's and Prim's algorithms to find Minimum Spanning Trees in graphs. The Kruskal's algorithm is implemented using an adjacency matrix, where edges are sorted by weight, and a union-find data structure is utilized to manage the connected components of the graph. Alongside finding the MST, the implementation also includes a mechanism to identify the second-best MST by evaluating alternative edges.

In contrast, Prim's algorithm is implemented using an adjacency list representation, leveraging a priority queue to efficiently select the edge with the minimum weight connecting the growing MST. The implementation additionally calculates the number of distinct MSTs by analyzing the weights of edges in the constructed MST and determining permutations based on shared weights.

By exploring both algorithms and their characteristics, the project deepens the understanding of graph theory concepts and their practical applications in network design, optimization, and resource management. The insights gained from identifying the second-best MST and counting distinct MSTs further highlight the complexity and versatility of spanning trees in graph analysis.